

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

**«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»**

Кафедра дискретной математики и информационных технологий

**РАЗРАБОТКА МЕТОДИКИ ИССЛЕДОВАНИЯ
СООТВЕТСТВИЯ ОС LINUX ЗАДАЧАМ РЕАЛЬНОГО
ВРЕМЕНИ**

КУРСОВАЯ РАБОТА

студента 3 курса 321 группы
направления 09.03.01 — Информатика и вычислительная техника
факультета КНиИТ
Номеровского Максима Сергеевича

Научный руководитель
старший преподаватель

Н. Е. Тимофеева

Заведующий кафедрой
доцент, к. ф.-м. н.

Л. Б. Тяпаев

Саратов 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Основы операционных систем реального времени и архитектуры Linux	5
1.1 Операционные системы реального времени (RTOS)	5
1.1.1 Основная информация	5
1.1.2 Планировщик ОСРВ	6
1.2 Ядро Linux	8
1.2.1 Основные понятия	8
1.2.2 Ядро Linux в качестве ядра для ОСРВ	9
1.2.3 Поточковые прерывания и планирование в PREEMPT_RT	9
1.2.4 Дерево устройств	11
1.3 Загрузчик U-Boot	12
2 Разработка и тестирование системы реального времени на базе MangoPi MQ Pro	13
2.1 Постановка задачи	13
2.2 Компиляция ядра Linux	14
2.3 Создание образа системы для MangoPi MQ Pro	17
2.4 Создание программного обеспечения для тестирования системы реального времени	19
2.5 Сборка схемы и проверка работы	20
2.5.1 Измерение с осциллографом	21
2.5.2 Программное измерение	22
2.6 Анализ результатов	24
2.6.1 Влияние приоритезации задач	24
2.6.2 Сравнение средних задержек	25
2.6.3 Анализ наихудшего времени отклика	25
ЗАКЛЮЧЕНИЕ	26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	27
Приложение А Исходный код программы для Arduino	29
Приложение Б Исходный код программы для MangoPi MQ Pro	32
Приложение В Исходный код программы для ПК	36

ВВЕДЕНИЕ

Операционные системы реального времени (**ОСРВ**) играют ключевую роль в управлении критически важными встраиваемыми системами, где задержка реакции на событие может привести к фатальным последствиям или существенному снижению производительности [1]. Традиционно ядро Linux не рассматривалось как полноценное решение для задач жесткого реального времени. Однако развитие проекта и недавнее официальное включение функциональности **PREEMPT_RT** в конфигурацию основной ветки ядра открыло новые перспективы для использования Linux в подобных системах. На сегодняшний день существует острая потребность в формировании понятных и воспроизводимых методик оценки того, насколько хорошо собранная система справляется с детерминированной обработкой данных на аппаратном уровне.

Интеграция патчей реального времени непосредственно в основную кодовую базу ядра Linux делает процесс развертывания ОСРВ более стандартизированным, что требует актуальных эмпирических данных о поведении системы под различной нагрузкой [2]. Большинство существующих исследований и сформированных практик сосредоточены на архитектурах **ARM** и **x86**. Новизна данной работы заключается в разработке и практической проверке методики тестирования метрик реального времени на базе активно развивающейся архитектуры **RISC-V**. Это позволяет расширить понимание применимости современных открытых архитектур в задачах с жесткими временными ограничениями.

Целью курсовой работы является разработка методики исследования соответствия операционной системы Linux задачам реального времени на конкретной аппаратной платформе и проведения сравнительного анализа при использовании ядра с активированным параметром **PREEMPT_RT** и без него для решения простейших задач реального времени.

Для достижения поставленной цели в работе будут решены следующие задачи:

- Выбрать и изучить аппаратную платформу, на которой будет проводиться и тестироваться методика.
- Выполнить кросс-компиляцию ядра Linux с параметрами, удовлетворяющими техническим требованиям выбранной аппаратной платформы, в двух конфигурациях (с поддержкой реального времени и без нее).

- Подготовить загрузочный образ системы, включающий загрузчик **U-Boot**, скомпилированное ядро, дерево устройств и корневую файловую систему.
- Разработать программно-аппаратный комплекс для генерации тестовых импульсов, фиксации задержек отклика и автоматизированного сбора статистики.
- Собрать тестовый стенд и провести серию аппаратных и программных измерений метрик реального времени в различных конфигурациях, в том числе с имитацией сторонней нагрузки на процессор.

1 Основы операционных систем реального времени и архитектуры Linux

1.1 Операционные системы реального времени (RTOS)

1.1.1 Основная информация

Операционная система реального времени (**ОСРВ**) – это программа, которая обеспечивает планирование выполнения задач в строго определённые сроки, управляет системными ресурсами и предоставляет надёжную основу для разработки прикладного программного обеспечения. Прикладной код, созданный на базе ОСРВ, может быть весьма разнообразным: от простого приложения для цифрового секундомера до значительно более сложных систем навигации летательных аппаратов [1].

Главной частью любой ОСРВ является **ядро** – базовое управляющее программное обеспечение, которое предоставляет минимальный набор логических функций, алгоритмы планирования и управления ресурсами. Также в ОСРВ включают комбинацию различных модулей помимо ядра, например файловую систему, стеки сетевых протоколов и другие компоненты.

Большинство ядер ОСРВ содержат следующие компоненты:

- **Планировщик (Scheduler)** – присутствует в каждом ядре и реализует набор алгоритмов, определяющих, какая задача и в какой момент времени будет выполняться. К наиболее распространённым алгоритмам планирования относятся, например, циклическое планирование (*round-robin*) и вытесняющее планирование (*preemptive scheduling*).
- **Объекты (Objects)** – специальные конструкции ядра, которые помогают разработчикам создавать приложения для встраиваемых систем реального времени. К типичным объектам ядра относятся задачи, семафоры и очереди сообщений. Очереди сообщений – это буфероподобные структуры данных, которые могут использоваться для синхронизации, обеспечения взаимного исключения и обмена данными путём передачи сообщений между задачами.
- **Сервисы (Services)** – операции, выполняемые ядром над объектами, а также общие системные функции, такие как работа с таймерами, обработка прерываний и управление ресурсами.

Операционные системы реального времени делят на два типа – системы **жёсткого реального времени** и системы **мягкого реального времени**.

Основное различие систем жёсткого и мягкого реального времени можно описать так: система жёсткого реального времени *никогда не опоздает* с реакцией на событие, система мягкого реального времени *не должна опаздывать* с реакцией на событие.

Операционная система, которая может обеспечить требуемое время выполнения задачи реального времени даже в худших случаях, называется **операционной системой жёсткого реального времени**. Ситуация, в которой обработка событий происходит за время, большее предусмотренного, в системе жёсткого реального времени считается фатальной ошибкой. При возникновении такой ситуации операционная система прерывает операцию и блокирует её, чтобы, насколько возможно, не пострадала надёжность и готовность остальной части системы.

Система, которая может обеспечить требуемое время выполнения задачи реального времени в среднем, называется **операционной системой мягкого реального времени**. В системе мягкого реального времени задержка реакции считается восстановимой ошибкой, которая может привести к увеличению стоимости результатов и снижению производительности, но не является фатальной.

1.1.2 Планировщик ОСРВ

Планировщик является центральным компонентом любого ядра. Он предоставляет алгоритмы, необходимые для определения того, какая задача и в какой момент времени должна выполняться. Базовыми единицами, конкурирующими за время выполнения, выступают планируемые сущности. В большинстве ядер к ним относятся задачи и процессы. **Задача** представляет собой независимый поток выполнения с собственной последовательностью инструкций, тогда как **процесс** — это схожий объект, но с более строгой защитой памяти, что достигается ценой дополнительных накладных расходов на производительность и ресурсы [3].

Чтобы обеспечить выполнение нескольких таких сущностей одновременно, ядро использует механизм **многозадачности**. В однопроцессорных системах это достигается не за счёт истинной параллельности, а путём быстрого последовательного переключения между потоками выполнения, что создаёт иллюзию их одновременной работы. При этом планировщик строго следит за тем, чтобы каждая задача запускалась в отведённые ей сроки,

тогда как процедуры обработки прерываний (**ISR**) запускаются аппаратно согласно своим установленным приоритетам. Чем больше задач в системе, тем выше требования к процессору, поскольку растёт частота переключений контекста [3].

Контекст — это состояние регистров процессора, необходимое задаче для корректного выполнения. Каждый раз, когда планировщик передаёт управление от одной задачи к другой, происходит переключение контекста. Для его реализации ядро создаёт для каждой задачи блок управления (**ТСВ**), в котором хранится вся необходимая информация о её состоянии. Пока задача выполняется, её контекст динамически меняется и поддерживается в ТСВ; когда задача приостанавливается, её контекст «замораживается» в том же блоке.

При переключении ядро сохраняет регистры текущей задачи в её ТСВ, загружает сохранённые регистры новой задачи и передаёт ей управление. Впоследствии, когда планировщик снова вернётся к приостановленной задаче, она продолжит выполнение ровно с того места, где остановилась. Само время переключения контекста обычно невелико, однако слишком частые переключения создают значительные накладные расходы, поэтому архитектуру приложения следует проектировать так, чтобы минимизировать их количество [3].

Непосредственным исполнителем переключения контекста выступает **диспетчер** — модуль, входящий в состав планировщика и отвечающий за изменение потока управления. В любой момент выполнения ОСРВ поток управления проходит либо через прикладную задачу, либо через обработчик прерываний, либо через ядро. При вызове системных функций управление передаётся ядру, а при выходе из него диспетчер решает, какой задаче передать управление далее. Это не обязательно та же задача, которая инициировала вызов: выбор следующей задачи определяется алгоритмами планирования. Логика работы диспетчера зависит от источника входа в ядро. Если системный вызов совершает задача, диспетчер активируется после каждого вызова, координируя возможные изменения в состояниях задач. Если же вызов инициирован обработчиком прерываний, диспетчер игнорируется до полного завершения ISR, даже если освобождение ресурсов формально позволяет переключить контекст. Это гарантирует, что критичная процедура прерывания

выполнится без вмешательства со стороны задач, и лишь после её завершения диспетчер получит управление для передачи его наиболее подходящей задаче.

Конкретная логика выбора следующей задачи задаётся алгоритмом планирования. Большинство современных ядер поддерживают два основных подхода: вытесняющее планирование на основе приоритетов и циклическое (*round-robin*) планирование.

По умолчанию в системах реального времени почти всегда используется **вытесняющая приоритетная модель**. В ней каждой задаче присваивается уровень приоритета (обычно от 0 до 255, где 0 — наивысший), и процессор всегда отдаётся задаче с максимальным приоритетом среди готовых к выполнению. Если в момент работы задачи с низким приоритетом появляется более приоритетная, ядро немедленно сохраняет контекст текущей задачи и переключается на новую. Приоритеты могут изменяться динамически в ходе работы приложения, что позволяет системе гибко реагировать на внешние события, однако неосторожное использование этой возможности чревато инверсией приоритетов, взаимными блокировками и отказом системы [1].

Чистое **циклическое планирование**, при котором все задачи получают равные доли процессорного времени, в системах реального времени практически не применяется, так как игнорирует разную степень важности задач. Вместо этого его комбинируют с приоритетной моделью, используя механизм квантования времени. В такой схеме задачи с одинаковым приоритетом выполняются по кругу, каждый получая фиксированный временной квант. Счётчик отсчитывает такты системного таймера, и по истечении кванта задача перемещается в конец очереди, а её счётчик сбрасывается. Если задача вытесняется более приоритетной, её накопленное время кванта сохраняется и восстанавливается при возвращении к ней, что гарантирует справедливое распределение ресурсов без ущерба для срочных операций.

1.2 Ядро Linux

1.2.1 Основные понятия

Ядро Linux — ядро операционной системы, соответствующее стандартам POSIX, составляющее основу операционных систем семейства Linux, а также ряда операционных систем для мобильных устройств.

Его основные функции:

- Управление аппаратными ресурсами: распределение процессорного времени, управление памятью, координация доступа к устройствам ввода-вывода.
- Обеспечение безопасности и разграничение прав доступа между процессами и пользователями.
- Поддержка сетевых протоколов стека TCP/IP.
- Управление файловыми системами и обеспечение чтения/записи данных.
- Предоставление API системных вызовов для пользовательских программ.

1.2.2 Ядро Linux в качестве ядра для OSCPВ

По стандарту ядро Linux не является ядром для операционных систем реального времени. Однако с 20 сентября 2024 года в конфигурации ядра был добавлен параметр `PREEMPT_RT`, позволяющий использовать ядро Linux в качестве ядра для OSCPВ. До этого эта функциональная возможность была реализована в виде набора внешних патчей ядра.

Данная функциональная возможность ядра реализовывает поддержку системы как жёсткого, так и мягкого реального времени.

1.2.3 Потокосые прерывания и планирование в `PREEMPT_RT`

В стандартном ядре Linux обработка аппаратных прерываний разделена на две части: «верхнюю» (*top half*), которая выполняется немедленно в контексте прерывания с отключенными линиями запросов, и «нижнюю» (*bottom half*), откладываемую для последующей обработки (механизмы софтверных `softirq`, тасклетов и очередей задач). Недостатком такого подхода для систем жесткого реального времени является невозможность вытеснения «верхней» половины обработчика, что порождает непредсказуемые задержки (джиттер) для прикладных высокоприоритетных задач.

Интеграция параметра `PREEMPT_RT` принципиально меняет эту архитектуру. Практически все обработчики аппаратных прерываний принудительно преобразуются в специализированные потоки ядра — так называемые потоковые прерывания (*threaded IRQs*), которые в списке процессов отображаются как `irq/<num>-<name>`. Поскольку эти обработчики теперь выполняются в контексте регулярных потоков ядра, они могут быть временно вытеснены

планировщиком, а внутри них допускается использование блокировок, способных переводить поток в состояние сна (например, `rt_mutex`) [4].

Перевод прерываний в контекст потоков ядра требует явного и строгого управления их приоритетами со стороны разработчика. Для задач реального времени в Linux предусмотрены специальные политики (режимы работы) планировщика, функционирующие в рамках классов реального времени:

- `SCHED_FIFO` (*First-In, First-Out*) — планирование по принципу «первым пришел — первым обслужен». Поток с этой политикой выполняется до тех пор, пока он сам не заблокируется (например, в ожидании ввода-вывода), не освободит процессор добровольно или не будет вытеснен более приоритетным потоком реального времени.
- `SCHED_RR` (*Round-Robin*) — циклический режим, аналогичный `SCHED_FIFO`, но с ограничением по времени (кванту) непрерывного выполнения. По истечении кванта времени поток перемещается в конец очереди своего уровня приоритета.

В отличие от стандартной политики планирования `SCHED_OTHER` (или `SCHED_NORMAL`), использующей динамические приоритеты и значения `nice` (от -20 до +19), для реального времени (`SCHED_FIFO` и `SCHED_RR`) выделен статический диапазон приоритетов от 1 (наинизший RT-приоритет) до 99 (наивысший RT-приоритет) [4]. Планировщик Linux всегда выбирает для выполнения поток с наибольшим численным значением RT-приоритета.

Критически важным аспектом проектирования детерминированных встраиваемых систем является правильное распределение приоритетов между прикладным программным обеспечением реального времени и потоковыми прерываниями ядра (`irq`). По умолчанию в `PREEMPT_RT` потокам прерываний назначается средний RT-приоритет (обычно равный 50). Однако в условиях высокой вычислительной или сетевой нагрузки такая конфигурация приводит к инверсии приоритетов и росту задержек.

Необходимость назначения максимального приоритета (вплоть до 99) для конкретного IRQ-процесса, отвечающего за обслуживание целевого аппаратного интерфейса (например, шины ввода-вывода GPIO), обусловлена следующими факторами:

1. **Изоляция от фонового ввода-вывода:** Если в системе параллельно выполняются другие системные или аппаратные задачи (например, се-

тевой стек или дисковые операции), их потоковые прерывания при одинаковом уровне приоритета начнут конкурировать за процессорное время с целевым прерыванием, обрабатывающим входящий измерительный сигнал. Это вызовет недетерминированную задержку активации обработчика.

2. **Безусловное вытеснение прикладных задач:** Даже если управляющая пользовательская программа выполняется с высоким приоритетом (например, 98), она не сможет обработать новые данные до тех пор, пока ядро не завершит их низкоуровневое извлечение из контроллера. Назначение IRQ-потоку приоритета 99 гарантирует, что при поступлении сигнала с аппаратного стенда планировщик мгновенно прервет любой поток (включая саму прикладную RT-задачу) для критически важной регистрации факта события.

Таким образом, согласно методологии построения систем реального времени на базе Linux, иерархия приоритетов должна выстраиваться от аппаратного драйвера к прикладному коду: максимальный приоритет (99) выделяется для целевого потока `irq`, чуть меньший (например, 98) — для прикладного потока-ответчика, а все остальные системные прерывания и утилиты нагрузочного тестирования принудительно оставляются на более низких позициях шкалы реального времени или в пространстве `SCHED_OTHER` [4]. Такой подход позволяет минимизировать аппаратный джиттер и обеспечить жесткую предсказуемость времени отклика системы в наихудших сценариях.

1.2.4 Дерево устройств

Device Tree (DT) — это структура данных, описывающая аппаратную конфигурацию системы для операционной системы. Появление Device Tree решило важную архитектурную проблему: без него каждая плата требовала бы отдельной сборки ядра с жёстко зашитыми адресами регистров и описаниями периферии [5]. Благодаря Device Tree одно и то же бинарное ядро может запускаться на разных платах одного семейства: информация об аппаратуре передаётся ядру отдельным файлом в момент загрузки, а не компилируется внутри.

DTS (Device Tree Source) — человекочитаемый текстовый файл с расширением `.dts`, содержащий описание аппаратуры конкретной платы. Файлы с расширением `.dtsi` являются включаемыми заголовочными файлами.

DTB (Device Tree Blob) – бинарный скомпилированный файл `.dtb`, который загружается загрузчиком и передаётся ядру Linux в момент старта.

DTC (Device Tree Compiler) – компилятор, преобразующий DTS-файлы в DTB и обратно.

Основной источник `dts` – исходный код ядра Linux и репозитории разработчиков одноплатных компьютеров.

1.3 Загрузчик U-Boot

Загрузчик (bootloader) – это небольшая программа, которая запускается сразу после включения устройства и отвечает за базовую инициализацию аппаратной части платы, поиск и загрузку в память основной операционной системы, передачу управления этой операционной системе.

Для одноплатников самым распространённым загрузчиком является **U-Boot** [6].

Этапы загрузки операционной системы одноплатного компьютера:

1. **Boot ROM.** Процессор выполняет встроенный код базовой аппаратной инициализации и осуществляет поиск вторичного загрузчика на внешнем носителе.
2. **SPL (Secondary Program Loader).** Компактная версия U-Boot, предназначенная исключительно для инициализации контроллера оперативной памяти (**DRAM**), что необходимо для загрузки полноценного загрузчика.
3. **U-Boot Proper.** Основная версия загрузчика, инициализирующая периферийные интерфейсы и загружающая в память образы ядра Linux, файла дерева устройств (**DTB**) и компонента **OpenSBI**.
4. **OpenSBI.** Реализация спецификации **SBI**, работающая в привилегированном режиме **M-mode**. Выполняет финальную низкоуровневую инициализацию процессора и подготавливает аппаратное окружение для передачи управления ОС.
5. **Ядро Linux.** После переключения в режим **S-mode** ядро инициализирует устройства согласно DTB, монтирует корневую файловую систему и запускает первый пользовательский процесс `init`.

U-Boot использует стандарт **Extlinux Configuration** для определения параметров запуска ядра. Файл `extlinux.conf` размещается в директории `/extlinux/` на разделе **BOOT (FAT32)**.

2 Разработка и тестирование системы реального времени на базе MangoPi MQ Pro

2.1 Постановка задачи

Для разработки методики исследования соответствия ОС Linux задачам реального времени необходимо решить следующие задачи:

1. Выбрать аппаратную платформу для последующей оценки разрабатываемой методики.
2. Выполнить сборку ядра Linux с конфигурацией, удовлетворяющей требованиям выбранной аппаратной платформы.
3. Подготовить образ операционной системы, включающий системный загрузчик и корневую файловую систему (**rootfs**).
4. Разработать программное обеспечение для автоматизированного тестирования и сбора метрик о функционировании операционной системы реального времени (ОСРВ).
5. Подготовить тестовый стенд и провести экспериментальное тестирование с использованием различных вариантов конфигурации ОСРВ.

В качестве целевой аппаратной платформы для проведения исследований выбран одноплатный компьютер **MangoPi MQ Pro**, построенный на базе системы-на-кристалле (SoC) **Allwinner D1** [7].

Основные технические характеристики платы:

- Система-на-кристалле Allwinner D1 с процессором XuanTie C906 (архитектура **RISC-V**) с тактовой частотой до 1 ГГц.
- Оперативная память объёмом 1 ГБ типа DDR3/DDR3L.
- Порт USB-OTG типа Type-C для подключения к ПК и подачи питания.
- Порт USB-Host типа Type-C для подключения внешних периферийных устройств.
- 40-контактный разъём расширения GPI.
- 24-контактный разъём для подключения цифровых камер или Ethernet-модулей (DVP/RGMII).
- Разъём mini HDMI для вывода видеосигнала на внешние дисплеи.
- Слот для карт памяти формата microSD (TF), используемый в качестве основного загрузочного носителя.
- Встроенный комбинированный модуль беспроводной связи Wi-Fi и Bluetooth RTL8723DS.

- 20-контактный FPC-разъём для подключения дисплеев по интерфейсам DSI, CTP или LVDS.
- Контактные площадки для подключения внешнего аудиовыхода.
- Встроенная печатная антенна для модуля Wi-Fi/Bluetooth.
- Компактные габариты платы размером 6,5х3 см.

В качестве вспомогательного оборудования для генерации тестовых воздействий и проведения измерений будет использоваться микроконтроллерная плата **Arduino UNO R4 Minima** [8].

2.2 Компиляция ядра Linux

В рамках данного исследования для проведения сравнительного анализа метрик будут использованы две версии ядра Linux: с активированным параметром `PREEMPT_RT` (для поддержки режима жесткого реального времени) и без него.

На первом этапе необходимо подготовить исходный код ядра. Для проведения экспериментов выбрано ядро Linux версии 6.16 из репозитория проекта **ALT Linux** [9].

Клонирование исходного кода осуществляется с помощью системы контроля версий Git [10], использование которой представлено в листинге 1.

Листинг 1: Клонирование исходного кода ядра Linux

```
git clone https://git.altlinux.org/gears/k/kernel-source-6.16.git
```

Затем осуществляется переход в директорию с загруженными исходными текстами [11] как показано в листинге 2.

Листинг 2: Переход в рабочий каталог исходного кода

```
cd kernel-source-6.16
```

Поскольку инструментальная ЭВМ (хост-система) функционирует под управлением операционной системы **NixOS**, процесс подготовки окружения для сборки ядра описывается с учетом специфики декларативного управления пакетами данного дистрибутива. Развертывание сборочной среды на других операционных системах будет существенно отличаться.

Для формирования изолированного сборочного окружения в корневой директории проекта создается конфигурационный файл `shell.nix` с содержимым, представленным в листинге 3.

Листинг 3: Конфигурационный файл `shell.nix` для среды NixOS

```
{ pkgs ? import <nixpkgs> {} }:  
  
let  
  crossPkgs = pkgs.pkgsCross.riscv64;  
  
in  
  pkgs.mkShell {  
    nativeBuildInputs = [  
      pkgs.buildPackages.stdenv.cc  
      pkgs.gnumake  
      pkgs.pkg-config  
      pkgs.bison  
      pkgs.flex  
      pkgs.bc  
      pkgs.elfutils  
      pkgs.openssl  
      pkgs.ncurses  
      pkgs.rsync  
      pkgs.kmod  
      pkgs.qemu  
      crossPkgs.stdenv.cc  
    ];  
  
    shellHook = ''  
      export ARCH=riscv  
      export CROSS_COMPILE=riscv64-unknown-linux-gnu-  
      export HOSTCC=gcc  
    '';  
  }
```

При выполнении утилиты `nix-shell` пакетный менеджер автоматически загрузит необходимые зависимости и инициализирует переменные окружения, требуемые для кросс-компиляции под целевую архитектуру.

Далее для генерации базовой конфигурации ядра выполняется команда, представленная в листинге 4.

Листинг 4: Генерация базовой конфигурации ядра

```
make defconfig
```

Последующая тонкая настройка необходимых параметров осуществля-

ется через псевдографический интерфейс, вызываемый специальной командой, представленной в листинге 5. Интерфейс показан на рисунке 1.

Листинг 5: Запуск меню конфигурации ядра

```
make menuconfig
```

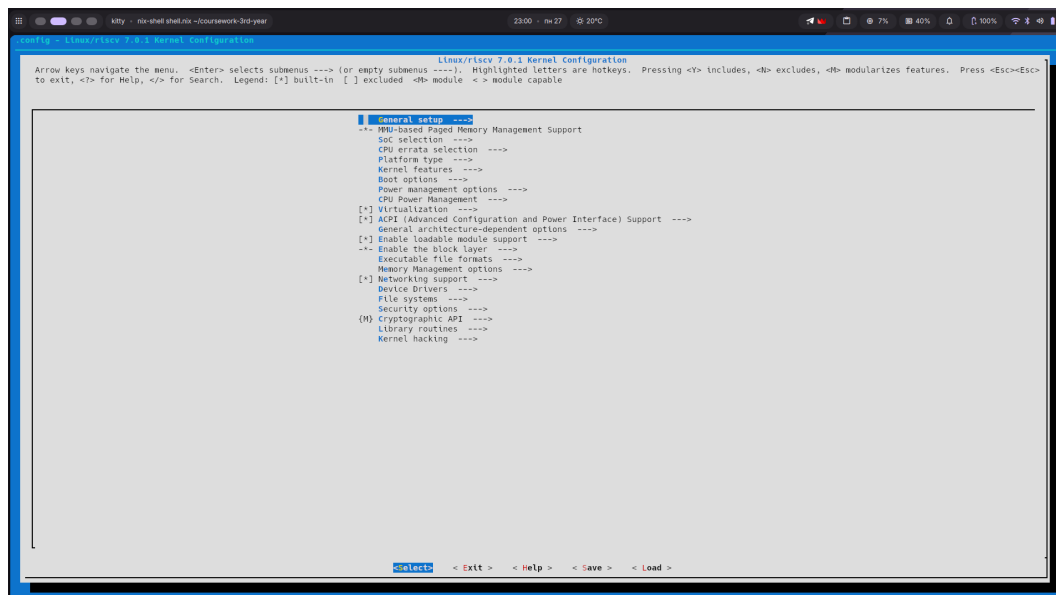


Рисунок 1 – Интерфейс меню конфигурации Linux

В главном меню утилиты конфигурации необходимо перейти в подраздел **SoC Selection**, где требуется деактивировать все доступные целевые платформы, за исключением **Allwinner sun20i SoCs**. Чтобы отключить конкретный пункт в списке, требуется установить на него курсор и нажать клавишу N.

Затем требуется активировать параметры конфигурации **EXPERT** и **PREEMPT_RT** [12]. Для нахождения данных параметров используется встроенная функция поиска, вызываемая клавишей \. Активация выбранных опций производится нажатием клавиши Y.

По завершении настройки измененная конфигурация сохраняется, после чего осуществляется выход из редактора.

Процесс многопоточной компиляции ядра инициируется следующей командой, показанной в листинге 6.

Листинг 6: Запуск многопоточной компиляции ядра Linux

```
make -j$(nproc)
```


После успешного завершения сборки из директории `./arch/riscv/boot` извлекается результирующий файл `Image`, представляющий собой исполняемый образ ядра Linux. Дополнительно из каталога `./dts/allwinner` копируется файл `sun20i-d1-mangopi-mq-pro.dtb` — скомпилированное дерево устройств (Device Tree Blob), необходимое для корректной аппаратной инициализации целевой платформы при загрузке.

Для подготовки второй версии системы (без поддержки жесткого реального времени) параметр `PREEMPT_RT` деактивируется, после чего процедура сборки выполняется повторно. На данном этапе из сборочной директории требуется извлечь исключительно обновленный файл образа ядра `Image`.

2.3 Создание образа системы для MangoPi MQ Pro

Для формирования полноценного образа операционной системы, помимо ядра, требуются системный загрузчик **U-Boot** и корневая файловая система (**rootfs**). В качестве загрузчика U-Boot используется сборка из репозитория проекта ALT Linux для архитектуры RISC-V [13]. Образ корневой файловой системы загружается с официального ресурса ALT Linux [14].

В рамках данной работы выбрана версия с графическим окружением **XFCE**.

После получения необходимых компонентов осуществляется подготовка карты памяти формата `microSD`, выступающей в роли загрузочного носителя. На начальном этапе производится очистка начальных секторов карты памяти путем записи нулевых байтов с использованием утилиты `dd` [15]. Пример использования показан в листинге 7.

Листинг 7: Очистка начальных секторов карты памяти

```
sudo dd if=/dev/zero of=/dev/sdX bs=1M count=10
```

Затем с помощью утилиты `fdisk` выполняется создание новой DOS-таблицы разделов и формирование двух логических разделов. Процесс создания описан в листинге 8.

Листинг 8: Разметка разделов карты памяти с помощью `fdisk`

```
sudo fdisk /dev/sdX # Заходим в утилиту

o                  # создаем новую DOS таблицу разделов
n                  # новый раздел
```

```

p                # primary
1                # номер раздела
2048             # начальный сектор
+100M           # размер 100MB
n                # новый раздел
p                # primary
2                # номер раздела
[Enter]          # начать сразу после первого раздела
+29.6G          # все оставшееся пространство
w                # записать изменения и выйти

```

Следующим шагом является форматирование созданных разделов. Раздел загрузчика U-Boot требует использования файловой системы **FAT** (vfat) [17], а для корневого раздела (второй раздел) выбрана журналируемая файловая система **ext4** [18]. Процесс форматирования представлен в листинге 9.

Листинг 9: Форматирование созданных разделов диска

```

sudo mkfs.vfat -n BOOT /dev/sdX1
sudo mkfs.ext4 -L ROOTFS /dev/sdX2

```

Запись загрузчика U-Boot на накопитель осуществляется следующей командой, представленной в листинге 10 [15].

Листинг 10: Запись загрузчика U-Boot на карту памяти

```

sudo dd if=/путь/до/загрузчика/u-boot-sunxi-with-spl.bin
\
of=/dev/sdX bs=1k seek=8

```

На первый раздел (BOOT) также копируются собранный ранее образ ядра Linux (Image) и бинарный файл дерева устройств (sun20i-d1-mangopi-mq-pro.dtb).

Для конфигурации параметров загрузки в корне первого раздела создается директория **extlinux**, содержащая конфигурационный файл **extlinux.conf** с содержимым, представленным в листинге 11.

Листинг 11: Конфигурационный файл extlinux.conf

```

label Linux
kernel /Image
fdt /sun20i-d1-mangopi-mq-pro.dtb
append root=/dev/mmcblk0p2 rootwait console=ttyS0,115200

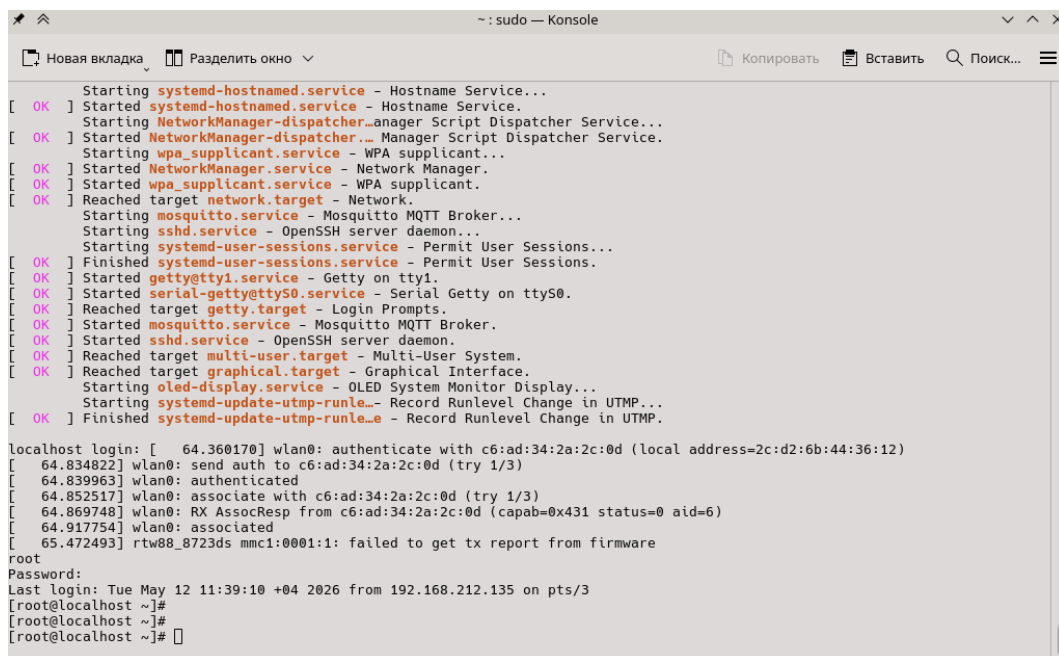
```

Заключительным этапом подготовки образа является распаковка архива корневой файловой системы (rootfs) на второй раздел карты памяти. Данная операция выполняется с помощью утилиты `tar` [11]. Использование утилиты показано в листинге 12.

Листинг 12: Распаковка корневой файловой системы на раздел ROOTFS

```
sudo tar -xJpf /путь/до/rootfs.tar.xz -C /путь/до_раздела
/ROOTFS/
```

После завершения распаковки образ системы считается готовым к использованию. Результат успешной загрузки операционной системы представлен на рисунке 2.



```
~: sudo — Konsole
[ OK ] Starting systemd-hostnamed.service - Hostname Service...
[ OK ] Started systemd-hostnamed.service - Hostname Service.
[ OK ] Starting NetworkManager-dispatcher.anag...
[ OK ] Started NetworkManager-dispatcher...
[ OK ] Starting wpa_supplicant.service - WPA supplican...
[ OK ] Started wpa_supplicant.service - WPA supplican...
[ OK ] Started NetworkManager.service - Network Manager.
[ OK ] Started wpa_supplicant.service - WPA supplican...
[ OK ] Reached target network.target - Network.
[ OK ] Starting mosquitto.service - Mosquitto MQTT Broker...
[ OK ] Started mosquitto.service - Mosquitto MQTT Broker.
[ OK ] Starting sshd.service - OpenSSH server daemon...
[ OK ] Started sshd.service - OpenSSH server daemon.
[ OK ] Starting systemd-user-sessions.service - Permit User Sessions...
[ OK ] Finished systemd-user-sessions.service - Permit User Sessions.
[ OK ] Started getty@tty1.service - Getty on tty1.
[ OK ] Started serial-getty@ttyS0.service - Serial Getty on ttyS0.
[ OK ] Reached target getty.target - Login Prompts.
[ OK ] Started mosquitto.service - Mosquitto MQTT Broker.
[ OK ] Started sshd.service - OpenSSH server daemon.
[ OK ] Reached target multi-user.target - Multi-User System.
[ OK ] Reached target graphical.target - Graphical Interface.
[ OK ] Starting oled-display.service - OLED System Monitor Display...
[ OK ] Started oled-display.service - OLED System Monitor Display.
[ OK ] Starting systemd-update-utmp-runle...
[ OK ] Finished systemd-update-utmp-runle...
[ OK ] Finished systemd-update-utmp-runle...
localhost login: [ 64.360170] wlan0: authenticate with c6:ad:34:2a:2c:0d (local address=2c:d2:6b:44:36:12)
[ 64.834822] wlan0: send auth to c6:ad:34:2a:2c:0d (try 1/3)
[ 64.839963] wlan0: authenticated
[ 64.852517] wlan0: associate with c6:ad:34:2a:2c:0d (try 1/3)
[ 64.869748] wlan0: RX AssocResp from c6:ad:34:2a:2c:0d (capab=0x431 status=0 aid=6)
[ 64.917754] wlan0: associated
[ 65.472493] rtw88_8723ds mmc1:0001:1: failed to get tx report from firmware
root
Password:
Last login: Tue May 12 11:39:10 +04 2026 from 192.168.212.135 on pts/3
[root@localhost ~]#
[root@localhost ~]#
[root@localhost ~]#
```

Рисунок 2 – Проверка работы собранного образа системы

2.4 Создание программного обеспечения для тестирования системы реального времени

Для тестирования разработанной системы используется программный комплекс, состоящий из трех компонентов:

1. **Генератор меандра для платформы Arduino.** Данное программное обеспечение генерирует прямоугольные импульсы длительностью 250 мкс и подает их на входной порт MangoPi MQ Pro. После инициации сигнала алгоритм ожидает ответного импульса от тестируемой платы и производит измерение времени задержки до изменения уровня ответного сигнала. Полученные телеметрические данные передаются

на персональный компьютер (ПК) для дальнейшей обработки. Листинг программы приведен в Приложении А.

2. **Программа-обработчик входящих импульсов.** Программа функционирует на целевой плате MangoPi MQ Pro и осуществляет опрос входного порта (GPIO) [19]. При регистрации логической единицы алгоритм генерирует ответный сигнал на назначенном выходном порту. Генерация ответного сигнала прекращается синхронно со снятием управляющего напряжения со стороны контроллера Arduino. Данный компонент обеспечивает базис для проведения временных измерений. Листинг программы приведен в Приложении Б.
3. **Программа сбора и анализа данных на ПК.** Приложение выполняется на ПК, принимает входящий поток данных от контроллера Arduino и производит вычисление средних значений задержек и показателей джиттера. Результаты вычислений экспортируются в файл формата .csv для удобного последующего анализа в табличных процессорах. Листинг программы приведен в Приложении В.

2.5 Сборка схемы и проверка работы

Для проведения дальнейших исследований осуществляется сборка аппаратного тестового стенда. В качестве порта для приема входного сигнала на плате MangoPi MQ Pro используется вывод PB10, для генерации выходного сигнала — вывод PB11. Распиновка отладочной платы представлена на рисунке 3.

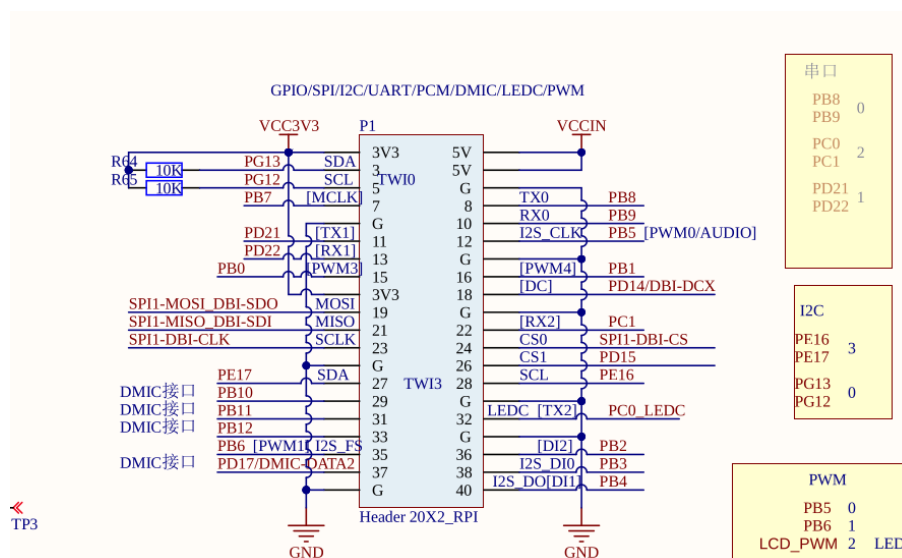


Рисунок 3 – Распиновка платы MangoPi MQ Pro

Соединение выводов между платами MangoPi MQ Pro и Arduino приведено в таблице 1.

Таблица 1 – Подключение выводов

MangoPi MQ Pro	Arduino
PB10	4
PB11	2
GND	GND

Для оценки степени соответствия ОС Linux требованиям систем жесткого реального времени проводятся два типа тестов: аппаратные (осциллографические) и программные измерения.

2.5.1 Измерение с осциллографом

На первом этапе производится ручное измерение характеристик сигналов с использованием осциллографа. Выполняется регистрация длительности задержек до сигнала (фронт) и после него (спад) для четырех различных конфигураций системы:

1. С использованием ядра Linux без параметра PREEMPT_RT.
2. С использованием ядра Linux без параметра PREEMPT_RT с наивысшим приоритетом у программы-ответчика.
3. С использованием ядра Linux с параметром PREEMPT_RT.
4. С использованием ядра Linux с параметром PREEMPT_RT с наивысшим приоритетом у программы-ответчика.

Результаты замеров приведены в таблице 2.

Таблица 2 – Результаты замеров осциллографом

Номер конфигурации	Задержка до сигнала(мкс)	Задержка после сигнала(мкс)
1	14	125
2	8	10
3	8	12.5
4	8.8	7.6

Анализ полученных данных показывает, что минимальная задержка реакции на получение входного сигнала наблюдается в конфигурациях 2 и 3. Однако наименьшая задержка реакции на прекращение сигнала зафиксирована в конфигурации 4. Таким образом, оптимальной конфигурацией с точки зрения совокупной скорости обработки является конфигурация 4.

В конфигурациях с повышенным приоритетом (варианты 2 и 4) прикладной процесс переводился в политику `SCHED_FIFO` с максимальным статическим приоритетом 99, что исключило конкуренцию программы-ответчика с системными задачами за процессорное время. Это привело к сокращению задержки по спаду сигнала со 125 мкс до 10 мкс на стандартном ядре и до 7.6 мкс на ядре с патчем реального времени.

2.5.2 Программное измерение

При проведении программных измерений используется ранее разработанное программное обеспечение для ПК. В перечень тестируемых конфигураций добавлены четыре дополнительных варианта, предполагающие использование утилиты `stress` [20] для искусственного нагружения центрального процессора платы и имитации побочной вычислительной нагрузки. Итоговый список конфигураций:

1. С использованием ядра Linux без параметра `PREEMPT_RT`.
2. С использованием ядра Linux без параметра `PREEMPT_RT` с утилитой `stress`.
3. С использованием ядра Linux без параметра `PREEMPT_RT` с наивысшим приоритетом у программы-ответчика.
4. С использованием ядра Linux без параметра `PREEMPT_RT` с наивысшим приоритетом у программы-ответчика с утилитой `stress`.
5. С использованием ядра Linux с параметром `PREEMPT_RT`.
6. С использованием ядра Linux с параметром `PREEMPT_RT` с утилитой `stress`.
7. С использованием ядра Linux с параметром `PREEMPT_RT` с наивысшим приоритетом у программы-ответчика.
8. С использованием ядра Linux с параметром `PREEMPT_RT` с наивысшим приоритетом у программы-ответчика с утилитой `stress`.

Для каждого варианта конфигурации была выполнена генерация и фиксация параметров для выборки, состоящей из 3000 сигналов. Общие сводные

данные, включающие минимальные, максимальные и средние значения длительности задержек, представлены в таблице 3.

Таблица 3 – Результаты программных замеров

№	Минимальная длительность задержки (мкс)	Максимальная длительность задержки (мкс)	Средняя длительность задержки (мкс)
1	194	1140	231.8840
2	183	4128	241.3326
3	157	331	193.5717
4	167	323	192.7687
5	219	2291	269.3970
6	192	5477	309.9773
7	167	248	203.6420
8	165	268	201.0947

Поскольку для систем жесткого реального времени критическое значение имеет гарантированное время реакции на событие, был проведен отдельный анализ предельных отклонений. В таблице 4 приведены статистические данные о пиковых значениях задержек (наихудшем времени отклика), зафиксированных в ходе тестирования. Эти показатели позволяют оценить детерминированность системы в стрессовых условиях.

Таблица 4 – Анализ пиковых значений задержек (наихудший случай)

№	Минимальная пиковая задержка (мкс)	Максимальная пиковая задержка (мкс)	Среднее значение пиковой задержки (мкс)
1	275	1140	360.2953
2	348	4128	805.0470
3	217	331	233.9267
4	210	323	215.5133
5	272	2291	633.2133
6	701	5477	1103.8658
7	223	248	226.9133
8	218	268	223.3200

Для изоляции измерительного процесса во всех конфигурациях с повышенным приоритетом (варианты 3, 4, 7 и 8) программа-ответчик запускалась в режиме реального времени **SCHED_FIFO** со статическим приоритетом 90. При этом сопутствующему целевому потоку обработки аппаратных прерываний ядра (**irq**) назначался максимальный приоритет 99.

Выстраивание такой иерархии, при которой приоритет низкоуровневого обработчика превосходит приоритет прикладного ПО, полностью согласуется с методологией проектирования предсказуемых встраиваемых систем [4]. Данная конфигурация необходима для того, чтобы при регистрации изменения сигнала на физическом уровне планировщик ядра имел возможность мгновенно вытеснить любые сторонние процессы (включая вычислительные потоки утилиты **stress**) и без задержек передать контекст процессора программе-ответчику.

2.6 Анализ результатов

На основе данных, полученных в ходе тестирования восьми конфигураций операционной системы, проведен анализ влияния архитектуры ядра, приоритезации процессов и фоновой нагрузки на временные характеристики.

2.6.1 Влияние приоритезации задач

Результаты показывают, что назначение наивысшего приоритета процессу-обработчику и **IRQ**-потоку (конфигурации 3, 4, 7 и 8) является клю-

чевым фактором снижения средних и пиковых задержек. В этих случаях планировщик ОС эффективно изолирует критически важный процесс от ресурсоемкой утилиты **stress**. Без повышения приоритета (конфигурации 2 и 6) фоновая нагрузка вызывает деградацию производительности: пиковая задержка возрастает до 4128 мкс (стандартное ядро) и 5477 мкс (**PREEMPT_RT**).

2.6.2 Сравнение средних задержек

Анализ средних значений выявил закономерность: ядро с параметром **PREEMPT_RT** обычно демонстрирует более высокие задержки по сравнению со стандартным. Этот эффект согласуется с теорией ОСРВ. Архитектурные особенности патча (перевод обработчиков прерываний в отдельные потоки и использование механизмов **rt-mutex**) создают накладные расходы на переключение контекста, что ожидаемо снижает общую пропускную способность.

2.6.3 Анализ наихудшего времени отклика

Оценку систем жесткого реального времени проводят по наихудшему времени реакции. Сравнение конфигураций с наивысшим приоритетом выявляет качественное преимущество **RT**-ядра.

Максимальная пиковая задержка стандартного ядра составила 331 мкс (без нагрузки) и 323 мкс (под нагрузкой). В то же время ядро **PREEMPT_RT** показало задержку 248 мкс (конфигурация 7) и 268 мкс под нагрузкой (конфигурация 8).

Таким образом, несмотря на накладные расходы, при правильной настройке приоритетов ядро **PREEMPT_RT** эффективнее ограничивает максимальное время отклика (минимизирует джиттер). Это является критически важным требованием для детерминированной обработки сигналов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была успешно решена задача разработки и практической проверки методики исследования операционной системы Linux на соответствие требованиям систем жесткого реального времени. В качестве аппаратной платформы выступил одноплатный компьютер MangoPi MQ Pro на базе активно развивающейся архитектуры RISC-V.

Для достижения поставленной цели был реализован полный цикл инженерно-исследовательских работ:

1. Осуществлена кросс-компиляция двух версий ядра Linux (стандартной и с интегрированным параметром `PREEMPT_RT`).
2. Подготовлен загрузочный образ операционной системы, включающий системный загрузчик U-Boot и корневую файловую систему.
3. Спроектирован и собран специализированный аппаратно-программный стенд с использованием микроконтроллера Arduino для высокоточной генерации тестовых импульсов и фиксации задержек отклика.

Результаты проведенных осциллографических и программных измерений полностью подтвердили теоретические принципы работы операционных систем реального времени. Было установлено, что наличие патча `PREEMPT_RT` не увеличивает среднюю пропускную способность системы; напротив, из-за дополнительных накладных расходов на переключение контекста (в том числе из-за перевода обработчиков прерываний в отдельные потоки) средняя длительность задержки может возрасть.

Ключевой характеристикой OCPV является предсказуемость в наихудших сценариях. Экспериментально доказано, что при жестком распределении приоритетов в режиме `SCHED_FIFO` (приоритет 99 для IRQ и 90 для программы) ядро реального времени эффективно изолирует процессы от фоновой нагрузки. В условиях стресс-тестирования конфигурация с `PREEMPT_RT` показала значительное преимущество над стандартным ядром Linux.

Разработанная и испытанная в ходе работы методика оценки временных метрик полностью доказала свою эффективность. Она является легко масштабируемой, воспроизводимой и может быть применена в будущем для дальнейшего детального профилирования, а также тонкой настройки множества других аппаратно-программных комплексов реального времени.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Qing Li, Caroline Yao. Real-time concepts for embedded system - CMP Books, 2003. - 320 с.
- 2 The Linux Kernel documentation [Электронный ресурс] URL: <https://docs.kernel.org/> (дата обращения: 27.04.2026)
- 3 SM Babamir. Real-Time Systems, Architecture, Scheduling, and Application - INTECH, 2012. - 352 с.
- 4 Симмондс К. Встраиваемые системы на основе linux - ДМК Пресс, 2017. - 360с.
- 5 Devicetree documentation [Электронный ресурс] URL: <https://docs.kernel.org/devicetree/usage-model.html> (дата обращения: 27.04.2026)
- 6 U-Boot documentation [Электронный ресурс] URL: <https://docs.u-boot.org/en/latest/> (дата обращения: 19.05.2026)
- 7 MangoPi MQ Pro documentation [Электронный ресурс] URL: https://mangopi.org/mangopi_mqpro (дата обращения: 27.04.2026)
- 8 Arduino documentation [Электронный ресурс] URL: <https://docs.arduino.cc/> (дата обращения: 19.05.2026)
- 9 AltLinux kernel source [Электронный ресурс] URL: <https://git.altlinux.org/gears/k/kernel-source-6.16.git> (дата обращения: 27.04.2026)
- 10 Git documentation [Электронный ресурс] URL: <https://git-scm.com/docs> (дата обращения: 27.04.2026)
- 11 William Shotts. The Linux Command Line - No Starch Press, 2026. - 544 с.
- 12 PREEMPT_RT setup guide [Электронный ресурс] URL: https://wiki.linuxfoundation.org/realtime/documentation/howto/applications/preemptrt_setup (дата обращения: 27.04.2026)
- 13 AltLinux U-Boot source [Электронный ресурс] URL: https://packages.altlinux.org/ru/sisyphus_riscv64/binary/u-boot-sunxi-riscv/noarch/3229656311355752582 (дата обращения: 27.04.2026)

- 14 AltLinux rootfs source [Электронный ресурс] URL:
<https://en.altlinux.org/Regular/riscv64> (дата обращения: 27.04.2026)
- 15 dd utility manual page [Электронный ресурс] URL:
<https://man7.org/linux/man-pages/man1/dd.1.html> (дата обращения: 27.04.2026)
- 16 fdisk manual page [Электронный ресурс] URL:
<https://man7.org/linux/man-pages/man8/fdisk.8.html> (дата обращения: 27.04.2026)
- 17 FAT filesystem documentation [Электронный ресурс] URL:
<https://docs.kernel.org/filesystems/vfat.html> (дата обращения: 27.04.2026)
- 18 Ext4 filesystem documentation [Электронный ресурс] URL:
<https://docs.kernel.org/filesystems/ext4/index.html> (дата обращения: 27.04.2026)
- 19 libgpio documentation [Электронный ресурс] URL:
<https://libgpiod.readthedocs.io/en/master/> (дата обращения: 27.04.2026)
- 20 stress utility manual page [Электронный ресурс] URL:
<https://linux.die.net/man/1/stress> (дата обращения: 19.05.2026)

ПРИЛОЖЕНИЕ А

Исходный код программы для Arduino

```
// Пины для подключения к MangoPi
const int PIN_OUT = 4;
const int PIN_IN  = 2;

// Временные интервалы
const unsigned long PULSE_DURATION_US = 500;
const unsigned long TIMEOUT_US        = 5000;

// Количество измерений
const int LATENCY_COUNT = 2;

// Времена фронтов
volatile unsigned long edgeTimes[LATENCY_COUNT];
volatile int edgeIndex = 0;

unsigned long pulseTimes[LATENCY_COUNT];

unsigned long count = 1;

// Прерывание по обоим фронтам
void onSignalChange() {
    if (edgeIndex < LATENCY_COUNT) {
        edgeTimes[edgeIndex++] = micros();
    }
}

void setup() {
    Serial.begin(115200);

    pinMode(PIN_OUT, OUTPUT);
    pinMode(PIN_IN, INPUT);

    digitalWrite(PIN_OUT, LOW);

    attachInterrupt(
        digitalPinToInterrupt(PIN_IN),
        onSignalChange,
        CHANGE
    );
}
```

```

        delay(2000);
    }

    void loop() {
        bool isTimeout = false;

        // Сброс состояния
        noInterrupts();
        edgeIndex = 0;
        interrupts();

        // Первый фронт
        pulseTimes[0] = micros();
        digitalWrite(PIN_OUT, HIGH);

        // Держим импульс
        while (micros() - pulseTimes[0] < PULSE_DURATION_US) {
        }

        // Второй фронт
        pulseTimes[1] = micros();
        digitalWrite(PIN_OUT, LOW);

        // Ждем оба фронта ответа
        unsigned long waitStart = micros();

        while (edgeIndex < LATENCY_COUNT) {
            if (micros() - waitStart > TIMEOUT_US) {
                isTimeout = true;
                break;
            }
        }

        // Вывод
        if (isTimeout) {
            Serial.print("ERROR;Timeout;");
            Serial.println(count);
        }
        else {
            Serial.print("DATA;");

```

```

        Serial.print(count);

        for (int i = 0; i < LATENCY_COUNT; i++) {
            long latency = edgeTimes[i] - pulseTimes[
                i];

            Serial.print(";");
            Serial.print(latency);
        }

        Serial.println();
    }

    count++;

    delay(1);
}

```

ПРИЛОЖЕНИЕ Б

Исходный код программы для MangoPi MQ Pro

```
#include <gpiod.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

#define CONSUMER_NAME "pulse_handler"
#define CHIP_PATH "/dev/gpiochip0" // Путь к устройству
#define LINE_IN_OFFSET 42          // Локальный номер входного
    пина
#define LINE_OUT_OFFSET 43         // Локальный номер выходного
    пина

int main(void) {
    struct gpiod_chip *chip = NULL;
    struct gpiod_line_settings *settings_in = NULL;
    struct gpiod_line_settings *settings_out = NULL;
    struct gpiod_line_config *line_cfg = NULL;
    struct gpiod_request_config *req_cfg = NULL;
    struct gpiod_line_request *request = NULL;
    struct gpiod_edge_event_buffer *event_buffer = NULL;

    unsigned int offset_in = LINE_IN_OFFSET;
    unsigned int offset_out = LINE_OUT_OFFSET;
    int ret_code = EXIT_SUCCESS;

    // 1. Открытие GPIO-контроллера
    chip = gpiod_chip_open(CHIP_PATH);
    if (!chip) {
        perror("Ошибка: Невозможно открыть указанный
            gpiochip");
        return EXIT_FAILURE;
    }

    // 2. Выделение памяти под структуры конфигурации
    settings_in = gpiod_line_settings_new();
    settings_out = gpiod_line_settings_new();
    line_cfg = gpiod_line_config_new();
    req_cfg = gpiod_request_config_new();
```



```

if (!settings_in || !settings_out || !line_cfg || !
    req_cfg) {
    fprintf(stderr, "Ошибка выделения памяти для
        структур конфигурации\n");
    ret_code = EXIT_FAILURE;
    goto cleanup;
}

// 3. Настройка линии ВВОДА (прерывание по обоим фронтам)
gpiod_line_settings_set_direction(settings_in,
    GPIOD_LINE_DIRECTION_INPUT);
gpiod_line_settings_set_edge_detection(settings_in,
    GPIOD_LINE_EDGE_BOTH);

// 4. Настройка линии ВЫВОДА (выход, начальное состояние
    - выключено)
gpiod_line_settings_set_direction(settings_out,
    GPIOD_LINE_DIRECTION_OUTPUT);
gpiod_line_settings_set_output_value(settings_out,
    GPIOD_LINE_VALUE_INACTIVE);

// 5. Упаковка настроек в общую конфигурацию линий
gpiod_line_config_add_line_settings(line_cfg, &offset_in,
    1, settings_in);
gpiod_line_config_add_line_settings(line_cfg, &offset_out
    , 1, settings_out);

// 6. Установка имени процесса, запросившего линии
gpiod_request_config_set_consumer(req_cfg, CONSUMER_NAME)
    ;

// 7. Аппаратный запрос линий с применением созданной
    конфигурации
request = gpiod_chip_request_lines(chip, req_cfg,
    line_cfg);
if (!request) {
    perror("Ошибка при аппаратном запросе линий");
    ret_code = EXIT_FAILURE;
    goto cleanup;
}

```

```

// 8. Создание буфера для приема событий прерываний из
    ядра (емкость 1 событие)
event_buffer = gpiod_edge_event_buffer_new(1);
if (!event_buffer) {
    perror("Ошибка создания буфера событий");
    ret_code = EXIT_FAILURE;
    goto cleanup;
}

printf("Обработчик запущен.\nОжидание прерываний на %s,
    линия %d...\n", CHIP_PATH, LINE_IN_OFFSET);

// 9. Основной цикл
while (1) {
    // Блокирующее ожидание аппаратного прерывания
    (-1 означает бесконечный таймаут).
    int wait_ret =
        gpiod_line_request_wait_edge_events(request,
        -1);

    if (wait_ret < 0) {
        perror("Сбой при ожидании аппаратного
            события");
        break;
    }

    // Чтение события из буфера драйвера
    if (gpiod_line_request_read_edge_events(request,
        event_buffer, 1) > 0) {
        // Получаем структуру конкретного события
        struct gpiod_edge_event *event =
            gpiod_edge_event_buffer_get_event(
                event_buffer, 0);
        enum gpiod_edge_event_type event_type =
            gpiod_edge_event_get_event_type(event);

        // Синхронная реакция на изменение
            сигнала
        if (event_type ==
            GPIOD_EDGE_EVENT_RISING_EDGE) {
            // Поступил импульс от

```

```

        контроллера -> выдаем 1 на
        выходной порт
        gpiod_line_request_set_value(
            request, offset_out,
            GPIOD_LINE_VALUE_ACTIVE);
    } else if (event_type ==
        GPIOD_EDGE_EVENT_FALLING_EDGE) {
        // Сигнал снят -> сбрасываем
        выходной порт в 0
        gpiod_line_request_set_value(
            request, offset_out,
            GPIOD_LINE_VALUE_INACTIVE);
    }
}

cleanup:
if (event_buffer) gpiod_edge_event_buffer_free(
    event_buffer);
if (request) gpiod_line_request_release(request);
if (req_cfg) gpiod_request_config_free(req_cfg);
if (line_cfg) gpiod_line_config_free(line_cfg);
if (settings_out) gpiod_line_settings_free(settings_out);
if (settings_in) gpiod_line_settings_free(settings_in);
if (chip) gpiod_chip_close(chip);

return ret_code;
}

```

ПРИЛОЖЕНИЕ В

Исходный код программы для ПК

```
import serial
import csv
import os

SERIAL_PORT = '/dev/ttyACM0'
BAUD_RATE = 115200
OUTPUT_FILE = 'arduino_statistics.csv'

if not os.path.exists(SERIAL_PORT):
    print(f"Ошибка: Порт {SERIAL_PORT} не найден.")
    exit(1)

# Переменные для расчета среднего за всё время сессии
total_count = 0
sum_start = 0
sum_end = 0
sum_dur = 0

need_tests = 5000

print(f"Подключение к {SERIAL_PORT}...")

try:
    ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)

    if os.path.exists(OUTPUT_FILE):
        print("Файл" + OUTPUT_FILE + "уже существует!")
        exit(0)

    with open(OUTPUT_FILE, mode='w', newline='', encoding='utf-8')
        as file:
            writer = csv.writer(file, delimiter=';')

            # Размечаем структуру таблицы в первой строке
            headers = [
                'Номер теста',
                'Задержка сигнала (мкс)',
                'СРЕДНЯЯ задержка сигнала (мкс)'
            ]
```

```

writer.writerow(headers)
file.flush()
print(f"Файл '{OUTPUT_FILE}' создан. Подписи столбцов внесены
      в 1-ю строку.")
print("Ожидаю данные от Arduino... (Нажмите Ctrl+C для
      остановки)")

while True:
    if ser.in_waiting > 0:
        line = ser.readline().decode('utf-8', errors='ignore').
            strip()

        if line.startswith("DATA;"):
            parts = line.split(';')
            data_row = parts[1:]

            # Извлекаем текущие показатели
            test_num = int(data_row[0])
            lat = int(data_row[1])

            # Инкрементируем метрики для глобального среднего
            total_count += 1
            sum_start += lat

            # Считаем среднее (актуальное на текущий тест)
            avg_start = sum_start // total_count

            # Собираем строку, где чередуются текущее значение и
            # среднее "за всё время"
            extended_row = [
                test_num,
                lat,
                avg_start
            ]

            # Записываем в CSV и пушим на жесткий диск
            writer.writerow(extended_row)
            file.flush()

            print(f"Успешно записан тест №{test_num} | Текущие
                  средние: ")

```

```

        f"Сигнал={avg_start}мкс")

    if need_tests == test_num:
        print("Успешно записали 5000 тестов, выходим")
        exit(0)

    elif line.startswith("DATA_ERROR;"):
        print("[!] Пропуск итерации: тайм-аут сигнала на
        стороне Arduino.")

except KeyboardInterrupt:
    print(f"\nСбор данных завершен. Всего тестов обработано: {
    total_count}. Файл сохранен.")
except PermissionError:
    print(f"\nОшибка прав доступа. Забыли сделать 'sudo chmod 666 {
    SERIAL_PORT}'?")
except Exception as e:
    print(f"\nКритическая ошибка: {e}")

```

Курсовая работа "Разработка методики исследования соответствия ОС Linux задачам реального времени" выполнена мною самостоятельно, и на все источники, имеющиеся в курсовой работе, даны соответствующие ссылки.

подпись, дата

инициалы, фамилия